

Task 1

Develop a multithreaded counter application using POSIX threads (pthread create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

Code:

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;

/* Function executed by each thread */
void *increase_counter(void *arg)
{
    int i;

    for (i = 0; i < 100000; i++)
    {
        counter++;
    }

    return NULL;
}

int main()
{
    pthread_t t1, t2, t3, t4;

    /* Create four threads */

    pthread_create(&t1, NULL, increase_counter, NULL);
    pthread_create(&t2, NULL, increase_counter, NULL);

    pthread_create(&t3, NULL, increase_counter, NULL);
    pthread_create(&t4, NULL, increase_counter, NULL);

    /* Wait for all threads */

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_join(t4, NULL);

    printf("Final counter value: %d\n", counter);

    printf("Expected value: 400000\n");

    return 0;
}
```

Output:

```
geya@LenovoL480:~/OS/Practical/Week 11$ ./task1
Final counter value: 180073
Expected value: 400000
geya@LenovoL480:~/OS/Practical/Week 11$ |
```

Task 2

Modify the previous program using mutex locks (pthread mutex lock, pthread mutex_unlock) and compare the outputs with the race- condition version. Analyze the impact of synchronization.

Code:

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;

/* Create mutex */
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

/* Function executed by each thread */
void *increase_counter(void *arg)
{
    int i;

    for (i = 0; i < 100000; i++)
    {
        /* Lock before accessing shared variable */

        pthread_mutex_lock(&lock);

        counter++;

        /* Unlock after accessing shared variable */

        pthread_mutex_unlock(&lock);
    }
}
```

```

    return NULL;
}

int main()
{
    pthread_t t1, t2, t3, t4;

    /* Create four threads */

    pthread_create(&t1, NULL, increase_counter, NULL);
    pthread_create(&t2, NULL, increase_counter, NULL);
    pthread_create(&t3, NULL, increase_counter, NULL);
    pthread_create(&t4, NULL, increase_counter, NULL);

    /* Wait for all threads */

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_join(t4, NULL);

    printf("Final counter value: %d\n", counter);
    printf("Expected value: 400000\n");
}

```

```

    /* Destroy mutex */

    pthread_mutex_destroy(&lock);

    return 0;
}

```

Output:

```

geya@LenovoL480:~/OS/Practical/Week 11$ ./task2
Final counter value: 400000
Expected value: 400000
geya@LenovoL480:~/OS/Practical/Week 11$ |

```